

NAME: DIANA NANSUBUGA

STUDENT ID:532026022

COURSE: ANALYSIS AND DESIGN OF ALGORITHMS

COURSE CODE: PRE 413

DATE: Oct 19, 2026

Contents

Summary **3**
1. Introduction **3**
2. Algorithms **3**
2.1 AlphaDev sort 4
2.2 PDQSort (pattern-defeating quicksort) 4
2.3 TwinArray Sort 5
2.4 Classic sorting algorithms 5
2.5 Search algorithms 5
3. Experimental setup **6**
3.1 How the measurements were made 6
4. Results **8**
4.1 Sorting at n = 1,000,000 8
4.2 Sorting at n = 100,000,000 10
4.3 Number of comparisons 12
4.4 Scaling from 1M to 100M 12
4.5 Memory usage 13
4.6 Searching 14
4.7 Bonus: parallel merge sort with OpenMP 15
5. Discussion **17**
5.1 There is no single best sorting algorithm 17
5.2 Theory predicts the work, not always the time 17
5.3 Modern versus classic algorithms 17
5.4 Fewer comparisons does not mean faster 17
5.5 Memory trade-offs 18
5.6 Searching 18
5.7 Parallel sorting 18
5.8 Limitations 18
6. Conclusion **19**
References **19**
Appendix A: Source files **20**

Summary

This project compares three modern sorting algorithms (AlphaDev sort, PDQSort and TwinArray Sort) with three classic ones (merge sort, heap sort and QuickSort) on four datasets (random, nearly sorted, reverse sorted and duplicates) at 1,000,000 and 100,000,000 integers. It also compares three search algorithms (binary search, exponential search and an M-way tree with M = 10) using 1000 random queries. For every run, execution time, number of comparisons and memory use were measured. As a bonus, merge sort was parallelised with OpenMP.

No single algorithm won everywhere. TwinArray Sort was fastest on random, nearly sorted and duplicate data, but needed 763 MB of extra memory at 100M and slowed down far more than its O(n + k) complexity predicts. PDQSort was by far the best on reverse-sorted data (189.51 ms at 100M against 887.99 ms or more for every other algorithm) and handled duplicates well. Classic QuickSort was the fastest comparison sort on random data, but became $O(n^2)$ on duplicates. Binary search was the best search algorithm overall. The OpenMP version of merge sort reached a 5.83× speedup with 20 threads and sorted 100M numbers faster than any serial algorithm.

1. Introduction

Sorting and searching are among the most frequently used operations in computing. Textbook analysis describes them with Big-O complexity, but real performance also depends on the input pattern, the size of the data and the hardware, especially the CPU cache and the number of cores. In recent years new algorithms have appeared that target these practical factors: PDQSort detects patterns in the input, and AlphaDev used reinforcement learning to discover faster small sorting routines that are now part of the LLVM C++ standard library.

The aim of this project is to test these algorithms at large scale and compare the measured behaviour with the theory. The project has five parts:

- **Sorting:** three modern and three classic sorting algorithms.
- **Datasets:** random, nearly sorted, reverse sorted and duplicates, at n = 1,000,000 and n = 100,000,000.
- **Searching:** binary search, exponential search and an M-way tree, each answering 1000 random queries.
- **Measurements:** execution time, number of comparisons and memory (extra heap memory and recursion depth).
- **Bonus:** parallel merge sort with OpenMP on a 20-thread CPU.

All programs were written in C++ by the author and are listed in Appendix A.

2. Algorithms

Table 1: *Algorithms used and their theoretical complexity (k = range of values)*

Algorithm	Group	Average time	Worst time	Extra memory
AlphaDev sort	Modern sort	O(n log n)	$O(n^2)$*	O(log n) stack
PDQSort	Modern sort	O(n log n)	O(n log n)	O(log n) stack
TwinArray Sort	Modern sort	O(n + k)	O(n + k)	O(k)
Merge sort	Classic sort	O(n log n)	O(n log n)	O(n)
Heap sort	Classic sort	O(n log n)	O(n log n)	O(1)
QuickSort	Classic sort	O(n log n)	$O(n^2)$	O(log n) to O(n) stack
Binary search	Search	O(log n)	O(log n)	O(1)
Exponential search	Search	O(log i)	O(log n)	O(1)
M-way tree (M = 10)	Search	$O(\log_{10} n)$ levels	$O(\log_{10} n)$ levels	O(n / 9) nodes

* The AlphaDev host here is a quicksort with median-of-three pivots. Its worst case is $O(n^2)$ in theory, but median-of-three and the balanced partition made it O(n log n) on all four datasets in practice. For exponential search, i is the position of the target.

2.1 AlphaDev sort

In 2023, DeepMind's AlphaDev system used reinforcement learning to search for shorter machine-code sequences for sorting 3, 4 and 5 elements. The routines it found (sort3, sort4 and sort5) were added to LLVM's libc++ std::sort. Their key property is that they are **sorting networks**: a fixed list of compare-and-swap steps with no branches, done with min and max.

In this project, AlphaDev sort is a quicksort that uses these networks. A median-of-three pivot is chosen, the array is split with a Hoare-style partition, and any part with 5 or fewer elements is finished by sort3, sort4 or sort5. The partition stops on values equal to the pivot from both sides, so many equal values are split evenly. Recursion goes to the smaller side and the larger side is handled by a loop, so the stack depth stays O(log n). This version reproduces AlphaDev's structure in C++; the exact assembly-level savings of the original depend on the compiler.

```
cmpSwap(x, y):  small = min(x, y); big = max(x, y); x = small; y = big
sort3(a, i):    cmpSwap(a[i],a[i+1]); cmpSwap(a[i+1],a[i+2]); cmpSwap(a[i],a[i+1])

alphaDev(a, lo, hi):
   loop:
      n = hi - lo
      if n <= 5: use sort2 / sort3 / sort4 / sort5 and return
      put median of a[lo], a[mid], a[hi-1] at a[lo]
      p = hoarePartition(a, lo, hi)      // equal values split evenly
      recurse on the smaller side, continue the loop on the bigger side
```

2.2 PDQSort (pattern-defeating quicksort)

PDQSort, by Orson Peters, is a quicksort that recognises patterns in the input. It is used in Go's standard sort and in Boost, and was the basis of Rust's sort_unstable. The simplified version in this project keeps its four main ideas:

- **Insertion sort** for parts smaller than 24 elements.
- **Duplicate handling:** if the new pivot equals the element just before the current part, all values equal to the pivot are moved to the left and skipped, so long runs of equal values are finished in one pass.
- **Sorted-input check:** if a partition did no swaps, the part may already be sorted, so a partial insertion sort is tried and abandoned after 8 moves if it fails.
- **Worst-case guard:** after $\log_2 n$ badly unbalanced partitions, the part is sorted with heap sort, which guarantees O(n log n).

Block partitioning and the pattern-breaking shuffles of the full PDQSort were left out to keep the code readable.

```
pdq(a, lo, hi, badAllowed):
   loop:
      if hi - lo < 24: insertionSort(a, lo, hi); return
      put median of three at a[lo] as the pivot
      if lo > 0 and a[lo-1] == pivot:          // many duplicates
         p = partitionLeft(a, lo, hi); lo = p + 1; continue
      p = partitionRight(a, lo, hi, noSwaps)
      if partition is very unbalanced:
         badAllowed -= 1; if badAllowed == 0: heapSort(a, lo, hi); return
      else if noSwaps and partialInsertionSort works on both sides: return
      pdq(a, lo, p, badAllowed); lo = p + 1
```

2.3 TwinArray Sort

TwinArray Sort is a non-comparison sort. It first finds the minimum and maximum value, then creates two arrays of size k = max − min + 1: value[] stores each number at its own index, and count[] stores how many times it

appears. The output is produced by walking through the arrays in order. The sorting step makes no comparisons at all; the only comparisons (2n) come from finding the minimum and maximum. The cost is memory proportional to k, so it only suits integers with a limited range.

```
twinArraySort(a):
    find mn and mx                  // 2n comparisons
    k = mx - mn + 1
    value[0..k-1] = 0; count[0..k-1] = 0
    for x in a: value[x - mn] = x; count[x - mn] += 1
    pos = 0
    for i in 0..k-1: repeat count[i] times: a[pos++] = value[i]
```

2.4 Classic sorting algorithms

- **Merge sort:** splits the array in half, sorts each half recursively and merges them using a temporary array of size n.
- **Heap sort:** builds a max-heap, then repeatedly swaps the largest element to the end and restores the heap. It sorts in place.
- **QuickSort:** textbook version with the middle element as pivot and the Lomuto partition, recursing on both sides. Values equal to the pivot all go to one side, which makes it $O(n^2)$ when there are many duplicates.

2.5 Search algorithms

- **Binary search:** compares the target with the middle element and keeps only the half that can contain it.
- **Exponential search:** checks positions 1, 2, 4, 8, … until it passes the target, then runs binary search inside the last jump. It is fast when the target is near the start of the array.
- **M-way tree (M = 10):** each node has up to 10 children and 9 keys. The tree is built bottom-up over the sorted array: the array is divided into blocks of 9 numbers, groups of 10 blocks get a parent node, groups of 10 nodes get a parent, and so on up to the root. Each key is the smallest value of a child, so the tree stores only these signposts and not a second copy of the data. M = 10 was chosen because a node (84 bytes) fits in two 64-byte cache lines and the tree is only about 8 levels deep at 100M.

```
mwaySearch(x):
    node = root
    loop:
        i = 0
        while i < count-1 and keys[i] <= x: i += 1      // choose child
        if node is at the bottom:
            scan block child[i] (9 numbers) for x and return found / not found
        node = child[i]
```

3. Experimental setup

Table 2: *Hardware and software*

Item	Details
CPU	Intel Core i7-12800H: 14 cores (6 performance + 8 efficiency), 20 threads
Operating system	Windows
Compiler	g++ (MSYS2), flags -O2 -std=c++17; OpenMP bonus also uses -fopenmp
Language	C++17

Table 3: *Datasets (n = 1,000,000 and 100,000,000)*

Dataset	How it is made
Random	n random integers from 0 to n − 1 (mt19937, seed 42)
Nearly sorted	The random numbers sorted, then 1% of positions swapped at random
Reverse sorted	The random numbers sorted from largest to smallest
Duplicates	n random integers from 0 to 999, so only 1000 different values

3.1 How the measurements were made

- **Data in memory:** each dataset is generated in memory with a fixed seed, so every run and every algorithm uses exactly the same numbers, and no time is spent reading files. Every algorithm sorts its own copy of the original array.
- **Execution time:** wall-clock time with std::chrono, averaged over 5 runs at 1M and 3 runs at 100M.
- **Comparisons:** every comparison goes through a counting function. Counting is done in a separate run, so it does not slow down the timed runs. Because the data and algorithms are deterministic, the comparison counts are identical on any computer, which makes them a hardware-independent measure.
- **Memory:** max depth is the deepest recursion level (stack memory). Extra (MB) is the extra heap memory: merge sort's temporary array, TwinArray's two arrays and the M-way tree's nodes.
- **Correctness:** every sorted output was checked with is_sorted(), and all three search algorithms had to find the same number of queries. Before the final runs, the search algorithms were also tested against the C++ library's binary_search on 150,000 queries.

- **Search timing:** one search takes only about 20 to 50 nanoseconds, which is too short to time on its own. Each query was therefore repeated 100 times and the time divided by 100. Each query was timed in 5 rounds and its fastest round was kept, so a random slowdown by the operating system does not appear as a worst case.
- **Fair conditions:** the laptop was plugged in and other programs were closed. An earlier run with MATLAB open in the background gave 100M timings that varied by up to two times between runs, so those results were discarded and all experiments were repeated.

QuickSort was not run on 100M duplicates. At 1M duplicates it already needed 512,870,978 comparisons, 20 times more than on random data, and at 100M the $O(n^2)$ behaviour would take hours.

4. Results

4.1 Sorting at n = 1,000,000

Table 4: *Sorting time (ms) at n = 1,000,000. Fastest per dataset in bold*

Algorithm	Random	Nearly sorted	Reverse sorted	Duplicates
AlphaDev sort	73.94	17.37	14.35	46.64
PDQSort	72.12	10.90	2.11	38.37
TwinArray Sort	16.34	10.17	9.85	2.21
Merge sort	96.32	25.05	25.94	71.76
Heap sort	118.77	64.52	57.93	94.54
QuickSort	66.72	25.65	20.68	246.44

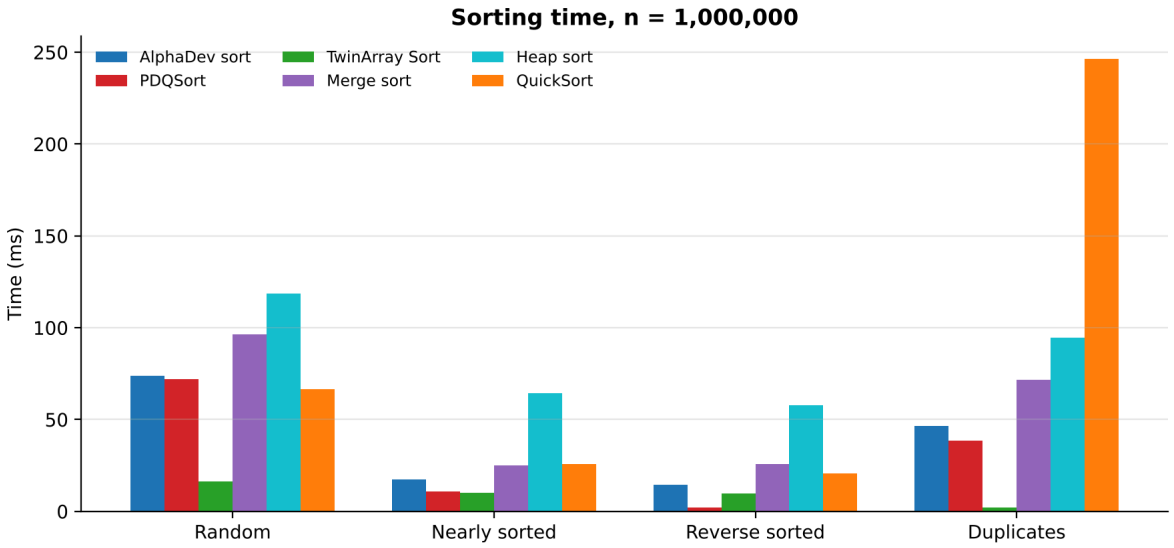


Figure 2: *Sorting time at n = 1,000,000*

Figure 3: *Program output: modern sorting algorithms, n = 1,000,000*

```
C:\Users\user\Documents\DAA-EJUST\PROJECT-ONE>classic_sort1m.exe

Dataset: Random, n = 1000000, time = average of 5 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-------------------------------------------------------------------------------
Merge sort             96.32        18674876          20          3.81       yes
Heap sort             118.77        36793933           0          0.00       yes
QuickSort              66.72        24880331          51          0.00       yes

Dataset: Nearly sorted, n = 1000000, time = average of 5 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-------------------------------------------------------------------------------
Merge sort             25.05        16437374          20          3.81       yes
Heap sort              64.52        37704186           0          0.00       yes
QuickSort              25.65        25593387          83          0.00       yes

Dataset: Reverse sorted, n = 1000000, time = average of 5 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-------------------------------------------------------------------------------
Merge sort             25.94        10263816          20          3.81       yes
Heap sort              57.93        36002153           0          0.00       yes
QuickSort              20.68        18582482          29          0.00       yes

Dataset: Duplicates, n = 1000000, time = average of 5 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-------------------------------------------------------------------------------
Merge sort             71.76        18670987          20          3.81       yes
Heap sort              94.54        36775940           0          0.00       yes
QuickSort             246.44       512870978        1112          0.00       yes
```

Figure 4: *Program output: classic sorting algorithms, n = 1,000,000*

4.2 Sorting at n = 100,000,000

Table 5: *Sorting time (ms) at n = 100,000,000. Fastest per dataset in bold*

Algorithm	Random	Nearly sorted	Reverse sorted	Duplicates
AlphaDev sort	9,853.18	2,034.64	1,690.93	4,924.99
PDQSort	10,157.52	1,474.29	189.51	3,828.35
TwinArray Sort	4,055.94	1,076.45	887.99	144.43
Merge sort	12,654.04	3,262.32	3,188.43	7,797.43
Heap sort	28,907.63	7,402.48	6,842.22	17,157.19
QuickSort	8,698.53	3,492.91	2,425.27	skipped ($O(n^2)$)

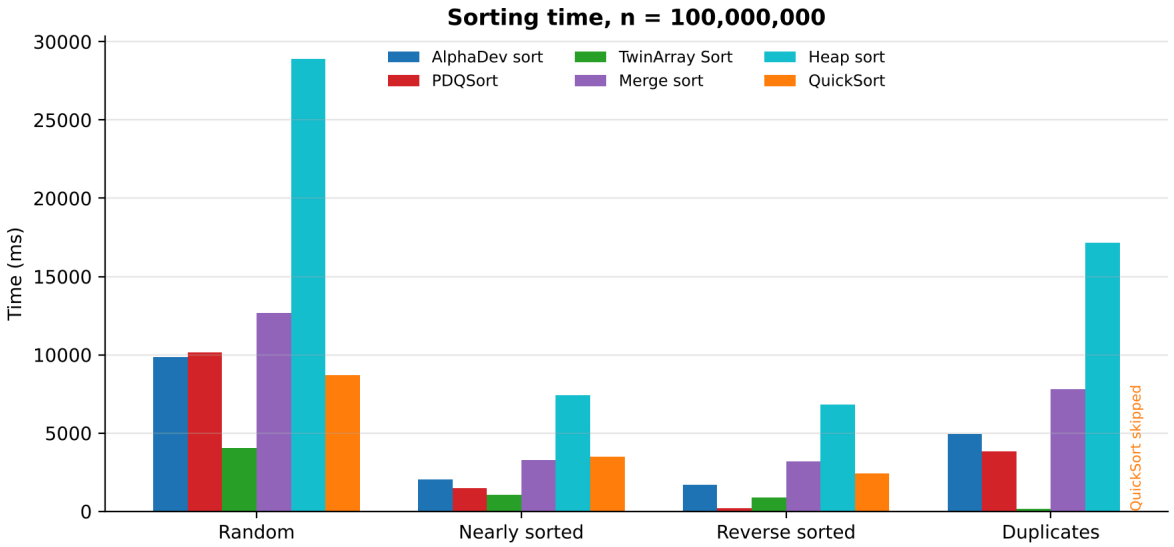


Figure 5: *Sorting time at n = 100,000,000*

```
Dataset: Random, n = 100000000, time = average of 3 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-------------------------------------------------------------------------------
AlphaDev sort        9853.18      2968822575          18          0.00       yes
PDQSort             10157.52      3104604878          35          0.00       yes
TwinArray Sort       4055.94       200000000           0        762.94       yes

Dataset: Nearly sorted, n = 100000000, time = average of 3 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-------------------------------------------------------------------------------
AlphaDev sort        2034.64      2641001849          23          0.00       yes
PDQSort              1474.29      2311483634          28          0.00       yes
TwinArray Sort       1076.45       200000000           0        762.94       yes

Dataset: Reverse sorted, n = 100000000, time = average of 3 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-------------------------------------------------------------------------------
AlphaDev sort        1690.93      2604863029          24          0.00       yes
PDQSort               189.51       300000001           1          0.00       yes
TwinArray Sort        887.99       200000000           0        762.94       yes

Dataset: Duplicates, n = 100000000, time = average of 3 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-------------------------------------------------------------------------------
AlphaDev sort        4924.99      2721581709          21          0.00       yes
PDQSort              3828.35      1239307200          11          0.00       yes
TwinArray Sort        144.43       200000000           0          0.01       yes
```

Figure 6: *Program output: modern sorting algorithms, n = 100,000,000*

```
C:\Users\user\Documents\DAA-EJUST\PROJECT-ONE>classic_sort100m.exe

Dataset: Random, n = 100000000, time = average of 3 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-----------------------------------------------------------------------------
Merge sort          12654.04      2532923647          27        381.47       yes
Heap sort           28907.63      5012885387           0          0.00       yes
QuickSort            8698.53      3389431225          67          0.00       yes

Dataset: Nearly sorted, n = 100000000, time = average of 3 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-----------------------------------------------------------------------------
Merge sort           3262.32      2303890896          27        381.47       yes
Heap sort            7402.48      5115790973           0          0.00       yes
QuickSort            3492.91      4145189294         190          0.00       yes

Dataset: Reverse sorted, n = 100000000, time = average of 3 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-----------------------------------------------------------------------------
Merge sort           3188.43      1364054534          27        381.47       yes
Heap sort            6842.22      4924225375           0          0.00       yes
QuickSort            2425.27      2522736699          44          0.00       yes

Dataset: Duplicates, n = 100000000, time = average of 3 runs
Algorithm          Time (ms)     Comparisons   Max depth    Extra (MB)    Sorted
-----------------------------------------------------------------------------
Merge sort           7797.43      2532187444          27        381.47       yes
Heap sort           17157.19      5009558141           0          0.00       yes
QuickSort        skipped: O(n^2) on duplicates, would take hours
```

Figure 7: *Program output: classic sorting algorithms, n = 100,000,000*

4.3 Number of comparisons

Table 6: *Comparisons at n = 1,000,000*

Algorithm	Random	Nearly sorted	Reverse sorted	Duplicates
AlphaDev sort	22,174,921	19,612,073	19,296,305	21,048,629
PDQSort	22,900,121	16,242,358	3,500,006	12,615,859
TwinArray Sort	2,000,000	2,000,000	2,000,000	2,000,000
Merge sort	18,674,876	16,437,374	10,263,816	18,670,987
Heap sort	36,793,933	37,704,186	36,002,153	36,775,940
QuickSort	24,880,331	25,593,387	18,582,482	512,870,978

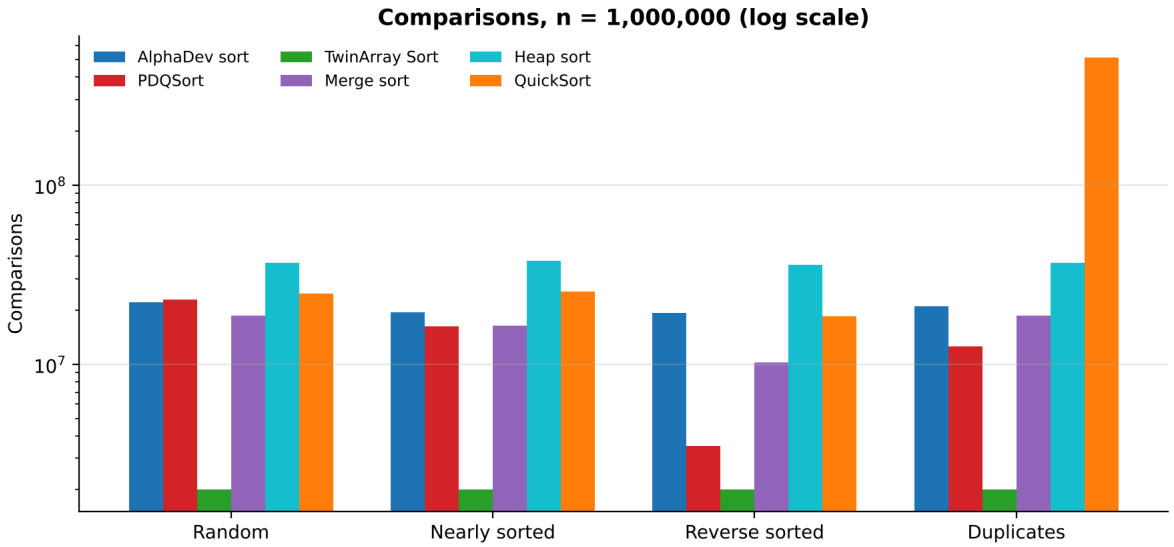


Figure 8: *Comparisons at n = 1,000,000 (log scale)*

The theoretical minimum for comparison sorting is about $\log_2(n!) \approx n \log_2 n - 1.44n$, which is 18.5 million at n = 1M. Merge sort (18.7 million) is very close to this bound on random data. QuickSort, AlphaDev and PDQSort need about 20 to 35% more, and heap sort about twice as many. The comparison counts at 100M are shown in Figures 6 and 7.

4.4 Scaling from 1M to 100M

When n grows 100 times, an O(n log n) algorithm should become $100 \times \log_2(10^8) / \log_2(10^6) \approx 133$ times slower, and an O(n + k) algorithm 100 times slower. Table 7 shows the measured growth on random data.

Table 7: *Growth in time and comparisons from 1M to 100M (random data)*

Algorithm	Time growth	Comparison growth	Theory	Matches theory?
AlphaDev sort	×133	×134	×133	Yes
PDQSort	×141	×136	×133	Yes
Merge sort	×131	×136	×133	Yes
QuickSort	×130	×136	×133	Yes
Heap sort	×243	×136	×133	No: time grew 1.8× more
TwinArray Sort	×248	×100	×100	No: time grew 2.5× more

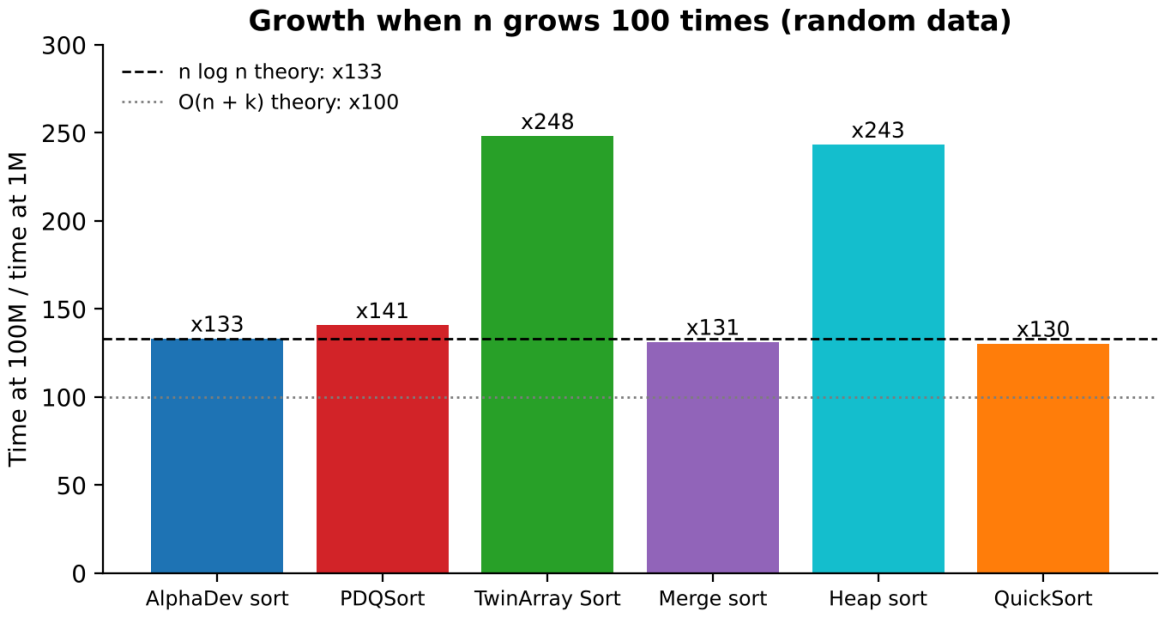


Figure 9: *Growth in time when n grows 100 times, compared with theory*

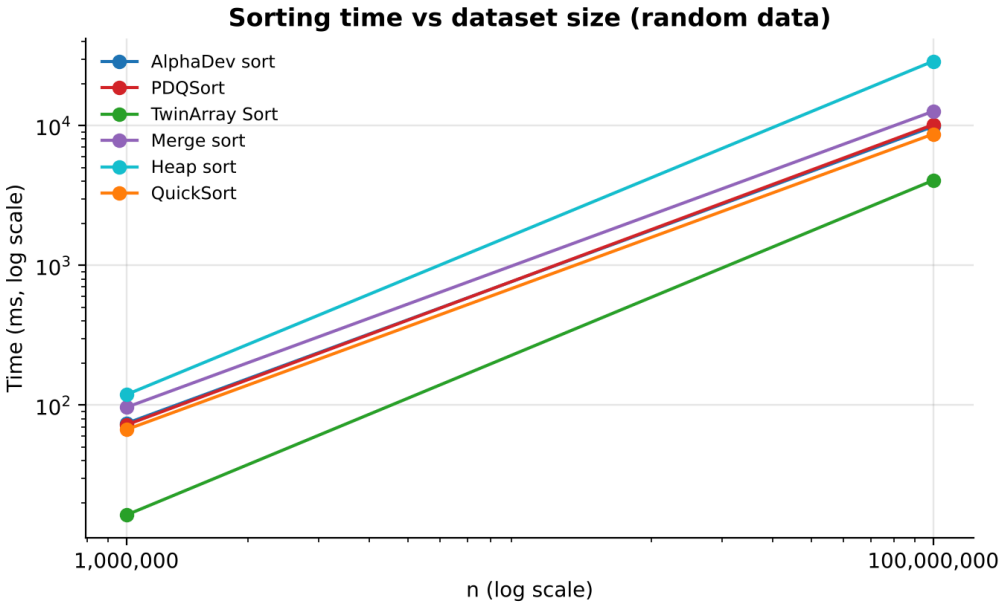


Figure 10: *Sorting time vs dataset size, random data (log scales)*

4.5 Memory usage

Table 8: *Extra heap memory and maximum recursion depth (random data)*

Algorithm	Extra MB, 1M	Extra MB, 100M	Max depth, 1M	Max depth, 100M
AlphaDev sort	0	0	13	18
PDQSort	0	0	22	35
TwinArray Sort	7.63	762.94	0	0
Merge sort	3.81	381.47	20	27
Heap sort	0	0	0	0
QuickSort	0	0	51	67

TwinArray's memory depends on the range of values, not on n: on the duplicates dataset (only 1000 different values) it needed just 0.01 MB. QuickSort's recursion went deepest, reaching 190 on nearly sorted data and 1,112 on 1M duplicates, because it recurses on both sides. AlphaDev sort and PDQSort keep the depth low by always recursing on the smaller side.

4.6 Searching

Table 9: *Search results for 1000 random queries*

Algorithm	n	Avg time (ns)	Worst time (ns)	Avg cmp	Worst cmp	Extra MB
Binary search	1M	19.0	23.0	37.4	40	0
Exponential search	1M	29.2	35.0	53.8	59	0
M-way tree (M = 10)	1M	29.2	42.0	36.8	60	0.99
Binary search	100M	29.3	35.0	50.8	54	0
Exponential search	100M	49.1	74.0	73.4	78	0
M-way tree (M = 10)	100M	38.2	59.0	47.8	75	98.90

All three algorithms found the same queries (646 of 1000 at 1M and 638 at 100M), which confirms they are correct. Building the M-way tree took 1.39 ms at 1M (12,351 nodes) and 152.91 ms at 100M (1,234,575 nodes); this is done once and is not included in the search times.

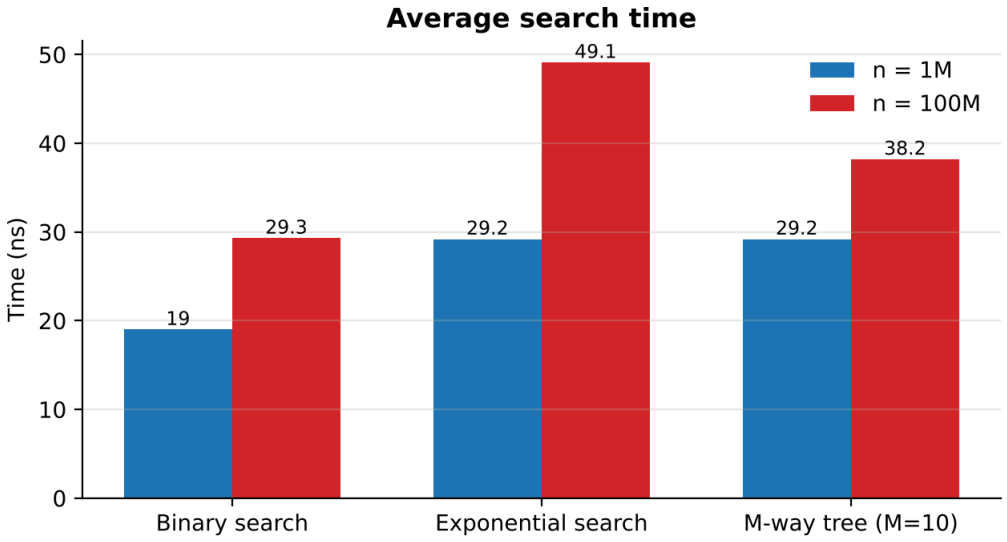


Figure 11: *Average search time*

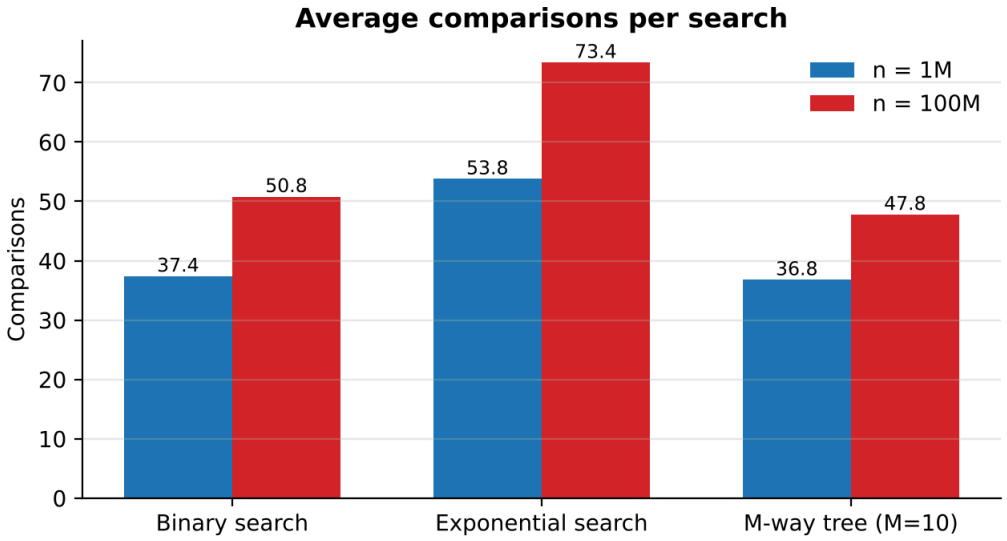


Figure 12: *Average comparisons per search*

```
C:\Users\user\Documents\DAA-EJUST\PROJECT-ONE>search.exe
Search on sorted random data, n = 1000000, 1000 queries
M-way tree: M = 10, 12351 nodes, built in 1.39 ms

Algorithm            Avg (ns) Worst (ns)   Avg cmp Worst cmp  Extra (MB)   Found
--------------------------------------------------------------------------------
Binary search            19.0       23.0      37.4        40        0.00     646
Exponential search       29.2       35.0      53.8        59        0.00     646
M-way tree (M=10)        29.2       42.0      36.8        60        0.99     646
```

Figure 13: *Program output: search, n = 1,000,000*

```
C:\Users\user\Documents\DAA-EJUST\PROJECT-ONE>search100m.exe
Search on sorted random data, n = 100000000, 1000 queries
M-way tree: M = 10, 1234575 nodes, built in 152.91 ms

Algorithm            Avg (ns) Worst (ns)   Avg cmp Worst cmp  Extra (MB)   Found
-------------------------------------------------------------------------------
Binary search            29.3       35.0      50.8        54        0.00     638
Exponential search       49.1       74.0      73.4        78        0.00     638
M-way tree (M=10)        38.2       59.0      47.8        75       98.90     638
```

Figure 14: *Program output: search, n = 100,000,000*

4.7 Bonus: parallel merge sort with OpenMP

Merge sort was parallelised with OpenMP tasks: the two halves are sorted as separate tasks that any free core can run, and parts smaller than 100,000 elements are sorted serially to avoid the cost of creating too many tasks. The same random data was used, and the time is the median of 3 runs.

Table 10: *Parallel merge sort: time and speedup*

Threads	Time 1M (ms)	Speedup 1M	Time 100M (ms)	Speedup 100M	Efficiency 100M
1	94.01	1.00×	12,477.39	1.00×	100%
2	50.01	1.88×	6,482.86	1.92×	96%
4	38.84	2.42×	4,519.18	2.76×	69%
8	25.94	3.62×	3,023.86	4.13×	52%
16	22.71	4.14×	2,296.24	5.43×	34%
20	22.16	4.24×	2,140.63	5.83×	29%

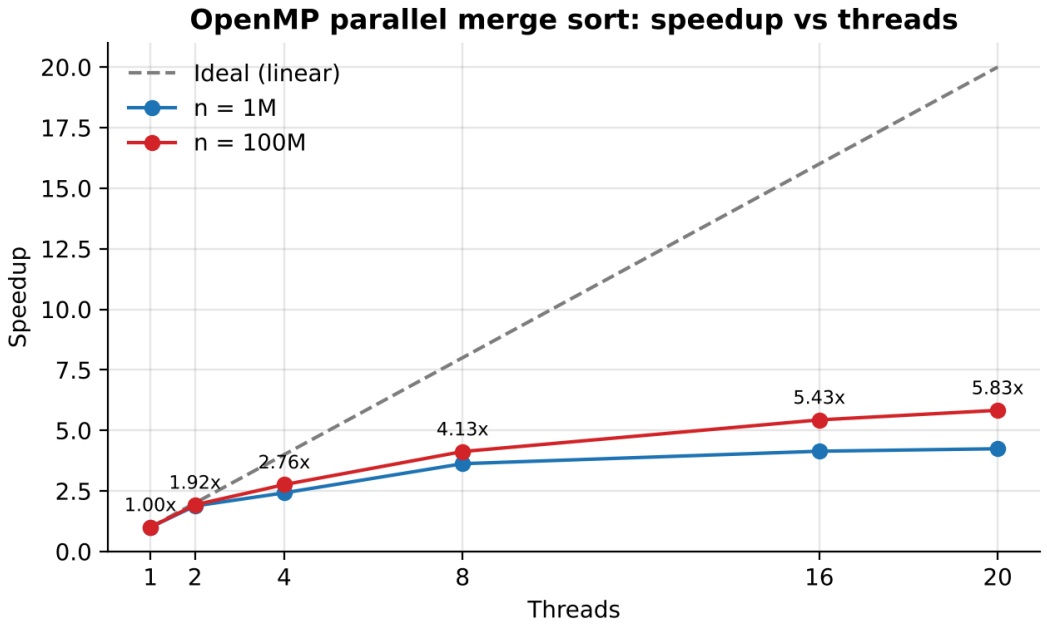


Figure 15: *Speedup of parallel merge sort compared with ideal linear speedup*

```
C:\Users\user\Documents\DAA-EJUST\PROJECT-ONE>pararell_sort.exe
Your CPU has 20 threads

Random data, n = 1000000, time = median of 3 runs
Threads        Time (ms)     Speedup    Sorted
-----------------------------------------------
1                 94.01       1.00x       yes
2                 50.01       1.88x       yes
4                 38.84       2.42x       yes
8                 25.94       3.62x       yes
16                22.71       4.14x       yes
20                22.16       4.24x       yes

Random data, n = 100000000, time = median of 3 runs
Threads        Time (ms)     Speedup    Sorted
-----------------------------------------------
1              12477.39       1.00x       yes
2               6482.86       1.92x       yes
4               4519.18       2.76x       yes
8               3023.86       4.13x       yes
16              2296.24       5.43x       yes
20              2140.63       5.83x       yes
```

Figure 16: *Program output: parallel merge sort*

5. Discussion

5.1 There is no single best sorting algorithm

The fastest algorithm depended on the dataset. TwinArray Sort won on random, nearly sorted and duplicate data at both sizes, PDQSort won on reverse-sorted data, and among the comparison sorts QuickSort was fastest on random data while PDQSort was fastest on every structured dataset at 100M. The best choice therefore depends on what the data looks like, how large it is, and how much memory is available.

5.2 Theory predicts the work, not always the time

The comparison counts followed theory closely: every comparison sort's count grew about 134 to 136 times from 1M to 100M, matching n log n (×133). For four algorithms the time grew the same way. Heap sort and TwinArray Sort did not: their time grew 243 and 248 times even though their work grew normally.

The reason is the memory hierarchy. At 1M, TwinArray's two arrays (7.6 MB) fit in the CPU cache, so every count[x]++ is fast. At 100M the arrays take 763 MB, and because the numbers are random, each access jumps to a random place in that array and has to wait for main memory. Heap sort has the same problem: it jumps between a parent at position i and a child at 2i + 1, which at 100M are far apart in memory. Merge sort, QuickSort, PDQSort and AlphaDev all read the array mostly from left to right, which the cache and hardware prefetcher handle well. Big-O counts steps, but it treats every memory access as equal, and at large scale they are not.

5.3 Modern versus classic algorithms

On random data, the modern comparison sorts had no advantage: classic QuickSort was the fastest comparison sort at both sizes (66.72 ms at 1M and 8,698.53 ms at 100M). Random data is QuickSort's best case, because the middle pivot almost always splits the array evenly, and its simple loop has less overhead than the extra checks in PDQSort and AlphaDev.

The value of the modern algorithms appeared on structured and difficult data:

- **Reverse-sorted data:** PDQSort needed only 3,500,006 comparisons at 1M (against 19,296,305 for AlphaDev) and finished 100M numbers in 189.51 ms, 13 times faster than classic QuickSort. After the first partition the data was already in order, so its sorted-input check stopped the work early. This shows the benefit of pattern detection.

- **Duplicates:** PDQSort made the fewest comparisons of all comparison sorts (12,615,859 at 1M) by skipping runs of equal values. Classic QuickSort with the Lomuto partition needed 512,870,978 comparisons, a recursion depth of 1,112, and could not be run at 100M at all. AlphaDev sort, whose partition splits equal values evenly, handled the same data normally.
- **Reliability:** AlphaDev sort never had a bad case and grew exactly as theory predicts (×133). Its sorting networks only handle the last few elements of each part, so its speed on random data is similar to a good quicksort, but its design avoids QuickSort's weaknesses.

5.4 Fewer comparisons does not mean faster

Merge sort made the fewest comparisons of the comparison sorts on random data (18,674,876 at 1M, close to the theoretical minimum), yet it was slower than QuickSort, AlphaDev and PDQSort. It spends time copying every element into the temporary array and back at each level, and needs 381 MB of extra memory at 100M. TwinArray Sort is the extreme case: almost no comparisons, but at 100M random its memory access pattern made it slower relative to its own 1M time than any other algorithm.

5.5 Memory trade-offs

The fastest sorting algorithm overall, TwinArray Sort, used the most memory (763 MB at 100M, almost twice the size of the data). Merge sort used as much extra memory as the data itself (381 MB). Heap sort used no extra memory at all, but was the slowest. QuickSort, AlphaDev and PDQSort sort in place with only a small stack, which makes them the best general-purpose choices when memory is limited.

5.6 Searching

Search time grows very slowly with n. The data grew 100 times, but binary search went from 37.4 to 50.8 comparisons on average, an increase of 13.4, close to the growth of $\log_2 n$ (+6.6) multiplied by binary search's two comparisons per step. Its time grew only 1.54 times.

Binary search was the best overall: fastest on average and in the worst case, with no extra memory. Exponential search was the slowest, because the queries were spread across the whole array; its doubling phase only pays off when the target is near the start, which is when it beats binary search. The M-way tree made the fewest comparisons on average, but each step to a new node is a memory access that costs more than a comparison, so it was slower than binary search. However, its time grew the least (×1.31 against ×1.54 for binary search): its upper levels are small and stay in the cache, while binary search's first steps jump across the whole 400 MB array. This is why databases use

B-trees for very large data, especially when it is stored on disk. Its costs are 99 MB of extra memory and a 153 ms build.

5.7 Parallel sorting

With 20 threads, parallel merge sort sorted 100M numbers in 2,140.63 ms. This was faster than every serial algorithm, including TwinArray Sort (4,055.94 ms), even though merge sort was one of the slowest serial sorts. Using the hardware made a larger difference than choosing a better algorithm.

The speedup (5.83×) was much lower than the ideal 20×. By Amdahl's law, $1/S = f + (1 - f)/p$, a speedup of 5.83 on 20 threads means that about 12.8% of the work behaves as if it were serial, which would limit the speedup to about 7.8× even with unlimited threads. The main serial part is the final merge of the two 50-million-element halves, which runs on a single core. In addition, the i7-12800H has only 6 performance cores; threads beyond these run on hyper-threads or on slower efficiency cores, which is why the curve flattens after 8 threads. All threads also share the same memory bandwidth. The 1M run scaled worse (4.24×) because each thread received very little work, so the cost of creating tasks was a larger share of the time.

5.8 Limitations

- All results come from one laptop. The comparison counts are hardware-independent, but the times will differ on other machines.
- Timing on a laptop is sensitive to background programs and CPU temperature; an earlier run with MATLAB open had to be discarded. The 100M results are averages of only 3 runs.
- PDQSort is a simplified version without block partitioning, and AlphaDev sort reproduces the structure of AlphaDev's routines in C++ rather than its exact machine code.
- Searching was tested only on the random dataset, with uniformly random queries.
- TwinArray Sort only works for integers with a limited range; it could not sort, for example, floating-point numbers or strings.

6. Conclusion

This project tested 6 sorting algorithms on 4 datasets at 1 million and 100 million integers, and 3 search algorithms with 1000 queries. The comparison counts followed Big-O theory closely, but the execution time also depended on how well each algorithm uses the CPU cache: the two algorithms with random

memory access, TwinArray Sort and heap sort, slowed down about twice as much as theory predicts at 100M.

TwinArray Sort was the fastest when its memory cost is acceptable. PDQSort was the strongest comparison sort, winning by a wide margin on reverse-sorted data and handling duplicates efficiently, which shows the value of pattern detection. Classic QuickSort was fast on random data but failed on duplicates. Binary search was the best search algorithm, while the M-way tree scaled best with size. Finally, parallelism gave the largest single improvement: with 20 threads, merge sort became faster than every serial algorithm.

Table 11: *Recommended algorithm for each situation*

Situation	Recommended	Reason
Integers with a small range of values	TwinArray Sort	No comparisons; very little memory when the range is small
General-purpose sorting	PDQSort	Fast on all data, detects patterns, O(n log n) guaranteed
Data that may be sorted or reversed	PDQSort	189.51 ms on 100M reverse-sorted data
Many duplicate values	PDQSort or TwinArray	Classic QuickSort becomes $O(n^2)$
Very little memory	Heap sort	No extra memory, O(n log n) guaranteed, but slowest
Many CPU cores available	Parallel merge sort	5.83× speedup with 20 threads
Searching a sorted array	Binary search	Fastest and no extra memory
Searching very large or on-disk data	M-way tree (B-tree)	Fewest comparisons, scales best with size

AI usage statement:

I used Claude (Anthropic), an AI assistant, during this project. Claude me debug the code and suggest better ways to write and debug the C++ programs for the sorting, search and OpenMP experiments, following my instructions on how the project should be structured (for example, generating the data in memory, using the same random numbers for every algorithm, and keeping the modern and classic algorithms in separate files). I compiled and ran every program on my own laptop, and all the results in this report are measured outputs from those runs, shown in the screenshots. When the 100M timings varied between runs, I identified that MATLAB was running in the background, closed it and repeated all the experiments. Claude also helped in the interpretation of the results, and drafted this report, which I then reviewed and edited. I checked that every number in the report matches my program outputs, and I take full responsibility for the final content.

References

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Spring Joint Computer Conference, 483–485.

Bayer, R., & McCreight, E. (1972). Organization and maintenance of large ordered indexes. Acta Informatica, 1(3), 173–189.

Bentley, J. L., & Yao, A. C. (1976). An almost optimal algorithm for unbounded searching. Information Processing Letters, 5(3), 82–87.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Mankowitz, D. J., et al. (2023). Faster sorting algorithms discovered using deep reinforcement learning. Nature, 618, 257–263.

OpenMP Architecture Review Board. OpenMP Application Programming Interface Specification. https://www.openmp.org

Peters, O. R. L. (2021). Pattern-defeating Quicksort. arXiv:2106.05123.

Appendix A: Source files

All programs are single C++ files. They generate their data in memory, so they need no input files.

Table 12: *Source files*

File	Contents	Compile command
modern_sort1M.cpp	AlphaDev, PDQSort, TwinArray; 4 datasets, n = 1M	g++ -O2 -std=c++17 modern_sort1M.cpp -o modern_1M.exe
modern_sort100M.cpp	Same, n = 100M	g++ -O2 -std=c++17 modern_sort100M.cpp -o modern_100M.exe
classic_sort1M.cpp	Merge, heap, QuickSort; 4 datasets, n = 1M	g++ -O2 -std=c++17 classic_sort1M.cpp -o classic_1M.exe
classic_sort100M.cpp	Same, n = 100M	g++ -O2 -std=c++17 classic_sort 100M.cpp -o classic_100M.exe
search.cpp	Binary, exponential, M-way tree; n = 1M	g++ -O2 -std=c++17 search.cpp -o search_1M.exe
search100M.cpp	Same, n = 100M	g++ -O2 -std=c++17 search100M.cpp -o search_100M.exe
pararell_sort.cpp	OpenMP parallel merge sort, 1M and 100M	g++ -O2 -std=c++17 -fopenmp parallel_sort.cpp -o paraell_sort.exe